

Software Engineering P1

2-Hour Rust Revision Exercises

Step by step practice - objectives, starter code, and hints

- Goal: revise the main concepts without going too deep into complex data structures.
- The goal is not to copy a solution, but to understand why Rust accepts or rejects code.
- Use the code help when you are stuck. It gives the shape, not the full final solution.
- Recommended command loop: cargo check -> cargo test -> cargo run.

Part	Time	Focus
Setup	10 min	Cargo project, modules, crate structure
Exercise 1	15 min	Mutability, ownership, borrowing
Exercise 2	15 min	Structs, enums, Option
Exercise 3	20 min	Result, ?, parsing, error handling
Exercise 4	20 min	Iterators, Vec, HashMap, HashSet
Exercise 5	20 min	Traits, default methods, generics
Exercise 6	15 min	Macros and modular main pipeline
Exercise 7	15 min	Unit tests, integration tests, coverage and performance commands

Global Mini-Project

Context: You build a small student data quality pipeline. It parses CSV-like lines, validates records, filters valid students, aggregates statistics, prints a report, and tests the behavior.

```
Input format:  
id,name,age,score,status
```

```
Example lines:  
1,Alice,23,88.5,active  
2,Bob,,71.0,inactive  
3,Charlie,19,,active  
bad,line  
4,Diana,200,91.0,active
```

Do not be stuck help: Keep the project small. The exercise is about concepts, not about building a perfect CSV parser.

Setup - Create the project

Objective: Prepare a clean Rust project with modules, like a real software engineering mini-project.

1. Create a new project named `revision_lab`.
2. Add a library root `src/lib.rs` so integration tests can import your code.
3. Create the modules: `models.rs`, `parsing.rs`, `filtering.rs`, `aggregation.rs`, `output.rs`, `macros.rs`.
4. Create tests/pipeline_test.rs for integration tests.

```
cargo new revision_lab  
cd revision_lab  
mkdir tests
```

```
touch src/lib.rs src/models.rs src/parsing.rs src/filtering.rs      src/aggregation.rs src/output.rs
src/macros.rs tests/pipeline_test.rs
```

```
// src/lib.rs
pub mod models;
pub mod parsing;
pub mod filtering;
pub mod aggregation;
pub mod output;
pub mod macros;
```

Do not be stuck help: If main.rs cannot see a module, remember: lib.rs is the library crate root. Integration tests import the project by crate name, for example use `revision_lab::parsing::parse_line`;

Exercise 1 - Fix ownership and borrowing bugs

Objective: Read short code snippets and fix them without changing the intended behavior.

Concepts revised: mutability, ownership move, borrowing, String vs &str

Steps

5. Create `src/ownership_drill.rs` or use a scratch file.
6. For each snippet, explain why it fails.
7. Fix it using either borrowing, cloning, or changing the order of operations.
8. Use cargo check after each fix.

Code help

```
// Snippet A - ownership move
let name = String::from("Alice");
let other_name = name;
println!("{}", name);

// Snippet B - immutable borrow then mutation
let mut message = String::from("hello");
let borrowed = &message;
message.push_str("!");
println!("{}", borrowed);

// Snippet C - function should not take ownership
fn print_name(name: String) {
    println!("{}", name);
}

let student = String::from("Diana");
print_name(student);
println!("{}", student);
```

Do not be stuck help: Pattern to remember: `fn print_name(name: &str)` can accept both `&String` and string literals. Use `&student` when you do not want to give ownership.

Expected result

- You can explain the difference between moving, borrowing, and cloning.
- Your fixed code compiles.
- You do not clone everything blindly: use references when possible.

Exercise 2 - Define the core data model

Objective: Create the struct and enum used by the pipeline.

Concepts revised: struct, enum, associated function, Option, impl block

Steps

9. In src/models.rs, define a Status enum with Active, Inactive, Unknown.
10. Define a StudentRecord struct with id, name, age, score, and status.
11. Use Option for missing age and missing score.
12. Add a constructor-like associated function StudentRecord::new.
13. Add small helper methods: is_active and display_name.

Code help

```
#[derive(Debug, Clone, PartialEq, Eq, Hash)]
pub enum Status {
    Active,
    Inactive,
    Unknown,
}

impl Status {
    pub fn from_str(value: &str) -> Self {
        match value.trim().to_lowercase().as_str() {
            "active" => Status::Active,
            "inactive" => Status::Inactive,
            _ => Status::Unknown,
        }
    }
}

#[derive(Debug, Clone, PartialEq)]
pub struct StudentRecord {
    pub id: u32,
    pub name: String,
    pub age: Option<u8>,
    pub score: Option<f32>,
    pub status: Status,
}

impl StudentRecord {
    pub fn new(id: u32, name: String, age: Option<u8>, score: Option<f32>, status: Status) -> Self {
        Self { id, name, age, score, status }
    }

    pub fn is_active(&self) -> bool {
        matches!(self.status, Status::Active)
    }
}
```

Do not be stuck help: Do not use null in Rust. Missing value = Option<T>. Example: age: Option<u8>.

Expected result

- A student without age or without score is representable.
- The enum avoids using random strings everywhere.
- Your model compiles and can be reused from other modules.

Exercise 3 - Parse input lines with Result and ?

Objective: Convert one CSV-like line into a StudentRecord while handling errors explicitly.

Concepts revised: Result, ? operator, Option, parse, error propagation

Steps

14. In src/parsing.rs, implement parse_optional_u8 and parse_optional_f32.
15. Implement parse_line(line: &str) -> Result<StudentRecord, String>.
16. Return Err if the line does not have exactly 5 fields.
17. Use ? to propagate parsing errors for id.
18. For missing age or score, return None instead of Err.

Code help

```
use crate::models::{Status, StudentRecord};

pub fn parse_optional_u8(input: &str) -> Option<u8> {
    let trimmed = input.trim();
    if trimmed.is_empty() {
        None
    } else {
        trimmed.parse::<u8>().ok()
    }
}

pub fn parse_line(line: &str) -> Result<StudentRecord, String> {
    let fields: Vec<&str> = line.split(',').collect();

    if fields.len() != 5 {
        return Err(format!("Invalid field count: {}", fields.len()));
    }

    let id = fields[0]
        .trim()
        .parse::<u32>()
        .map_err(|error| error.to_string())?;

    let name = fields[1].trim().to_string();
    let age = parse_optional_u8(fields[2]);
    let score = /* same idea as age, but f32 */;
    let status = Status::from_str(fields[4]);

    Ok(StudentRecord::new(id, name, age, score, status))
}
```

Do not be stuck help: Use .ok() when you want to convert Result<T, E> into Option<T>. Use map_err when the function returns Result<T, String>.

Expected result

- parse_line("1,Alice,23,88.5,active") returns Ok(StudentRecord).
- parse_line("bad,line") returns Err.
- Empty age or score becomes None.

Exercise 4 - Filter and aggregate records

Objective: Use iterators and collections to clean and summarize the dataset.

Concepts revised: Vec, iterators, filter, map, collect, HashMap, HashSet

Steps

19. In `src/filtering.rs`, implement `filter_valid(records: Vec<StudentRecord>) -> Vec<StudentRecord>`.
20. A valid record must have a non-empty name, `age <= 120` if age exists, and a known status.
21. In `src/aggregation.rs`, create `Summary` with `total_count`, `active_count`, `average_score`, `status_count`, and `unique_names`.
22. Use iterators for filtering and `HashMap/HashSet` for counting and uniqueness.

Code help

```
// src/filtering.rs
use crate::models::{Status, StudentRecord};

pub fn filter_valid(records: Vec<StudentRecord>) -> Vec<StudentRecord> {
    records
        .into_iter()
        .filter(|record| !record.name.trim().is_empty())
        .filter(|record| record.age.map_or(true, |age| age <= 120))
        .filter(|record| !matches!(record.status, Status::Unknown))
        .collect()
}

// src/aggregation.rs
use std::collections::{HashMap, HashSet};
use crate::models::{Status, StudentRecord};

#[derive(Debug, Default)]
pub struct Summary {
    pub total_count: usize,
    pub active_count: usize,
    pub average_score: Option<f32>,
    pub status_count: HashMap<Status, usize>,
    pub unique_names: HashSet<String>,
}
```

Do not be stuck help: `iter()` borrows. `into_iter()` consumes. `iter_mut()` mutably borrows. Choose based on whether you still need the original collection.

Expected result

- Invalid lines are removed before aggregation.
- The average ignores students with score = `None`.
- The `HashSet` contains each name only once.

Exercise 5 - Traits, default methods, generics, and lifetime mini-drill

Objective: Add reusable behavior without duplicating code.

Concepts revised: trait, default method, generic function, trait bound, lifetime

Steps

23. Create a `Validate` trait with `is_valid` and a default `validation_message` method.
24. Implement `Validate` for `StudentRecord`.
25. Create a `HasScore` trait and a generic function `has_score_above<T: HasScore>`.
26. Create `best_by_score` that returns a reference to the best student among two references.
27. Explain why the function needs a lifetime annotation.

Code help

```
pub trait Validate {
    fn is_valid(&self) -> bool;
```

```

    fn validation_message(&self) -> &'static str {
        if self.is_valid() { "valid" } else { "invalid" }
    }
}

pub trait HasScore {
    fn score_value(&self) -> Option<f32>;
}

pub fn has_score_above<T: HasScore>(item: &T, threshold: f32) -> bool {
    item.score_value().map_or(false, |score| score > threshold)
}

pub fn best_by_score<'a>(a: &'a StudentRecord, b: &'a StudentRecord) -> &'a StudentRecord {
    let a_score = a.score.unwrap_or(0.0);
    let b_score = b.score.unwrap_or(0.0);

    if a_score >= b_score { a } else { b }
}

```

Do not be stuck help: A lifetime does not make a value live longer. It tells Rust how returned references are related to input references.

Expected result

- Validation logic is callable as record.is_valid().
- The default method works without being rewritten every time.
- The generic function works for any type implementing HasScore.
- You can explain that the returned reference must be tied to input references.

Exercise 6 - Wire the modules and add a small macro

Objective: Integrate the small modules into a working pipeline and reduce logging boilerplate.

Concepts revised: module, pub, use, macro_rules!, pipeline design

Steps

28. In src/macros.rs, create a step_log! macro.
29. In src/output.rs, create a report(summary: &Summary) -> String function.
30. In main.rs, parse sample data, filter it, aggregate it, and print the report.
31. Keep main.rs readable: it should orchestrate, not contain all logic.

Code help

```

// src/macros.rs
#[macro_export]
macro_rules! step_log {
    ($($arg:tt)*) => {
        println!("[STEP] {}", format_args!($($arg)*));
    };
}

// src/main.rs
use revision_lab::parsing::parse_line;
use revision_lab::filtering::filter_valid;
use revision_lab::aggregation::summarize;
use revision_lab::output::report;
use revision_lab::step_log;

```

```
fn main() {
    let input = "1,Alice,23,88.5,active
2,Bob,,71.0,inactive";

    step_log!("Parsing records");
    // collect only successful records here

    step_log!("Filtering records");
    // call filter_valid

    step_log!("Creating summary");
    // call summarize and report
}
```

Do not be stuck help: Common module rule: mod declares a module. pub exposes it. use only brings a name into scope.

Expected result

- main.rs shows the big pipeline steps clearly.
- Modules hide implementation details.
- The macro logs repeated step messages with one clean syntax.

Exercise 7 - Test it like software engineering

Objective: Write tests that prove behavior instead of only hoping the code works.

Concepts revised: unit test, integration test, assert_eq, assert, cargo test, coverage, benchmarking

Steps

32. Add unit tests inside parsing.rs for parse_line.
33. Add unit tests inside filtering.rs for invalid age and unknown status.
34. Add an integration test in tests/pipeline_test.rs that uses the public API.
35. Run cargo test.
36. Write down which command you would use for coverage and benchmarking.

Code help

```
// inside a module file
#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn parses_valid_line() {
        let record = parse_line("1,Alice,23,88.5,active").unwrap();
        assert_eq!(record.name, "Alice");
        assert_eq!(record.age, Some(23));
    }
}

// tests/pipeline_test.rs
use revision_lab::parsing::parse_line;

#[test]
fn public_api_parses_line() {
    let record = parse_line("1,Alice,23,88.5,active").unwrap();
    assert!(record.is_active());
}

// Commands to know:
```

```
// cargo test
// cargo test test_name
// cargo test -- --nocapture
// cargo tarpaulin --out Html
// cargo bench
```

Do not be stuck help: Good tests are small, independent, readable, and focused on behavior. Avoid testing private implementation details from the outside.

Expected result

- At least 5 tests pass.
- One test checks a successful case.
- One test checks an invalid or edge case.
- One test is an integration test through the public API.

Optional Bonus

- Add a simple command-line argument for the input file path using `std::env::args`.
- Use a `BTreeMap` instead of `HashMap` in the report if you want deterministic sorted output.
- Add a small benchmark later with criterion to compare two aggregation implementations.
- Concept question: where would a smart pointer be useful in a larger version of this project? Example: shared immutable config could use `Arc` in a multithreaded version.

Final 2-Hour Checklist

Concept	Can the student do it?
Cargo and modules	Project compiles with several files and clear pub/use paths
Ownership and borrowing	Can fix moved value and borrow conflict errors
Structs and enums	Can model data with clear types
Option and Result	Can represent missing values and errors explicitly
?	Can propagate parsing errors cleanly
Iterators	Can use filter/map/collect/fold-like logic
Collections	Can choose Vec, HashMap, HashSet for simple needs
Traits and generics	Can write reusable behavior with trait bounds
Lifetimes	Can explain returned references tied to inputs
Macros	Can use macro_rules! for small boilerplate reduction
Testing	Can write unit and integration tests
Performance/reliability	Knows cargo test, cargo tarpaulin, cargo bench, cargo clippy

Do not solve everything for them. Ask first: Who owns this value? Is this an absent value or an error? Am I borrowing or consuming? Which module should know this logic?